# **Guía de** Actividades **PrácticoExperimentales** Nro. 002

## 1. Datos Generales

| | |
|---|---|
| **Asignatura** | Diseño Digital |
| **Ciclo** | 5to |
| **Unidad** | Uno |
| **Resultado de aprendizaje de la unidad** | R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 2 |
| **Título de la Práctica** | Juego de Luces Interactivo |
| **Nombre del Docente** | José O. Guamán Q. |
| **Nombre de Estudiante** | Viviana E. Córdova C. |
| **Fecha** | 14-04-2026<br>16-04-2026 |
| **Horario** | Martes 12h30 – 13h30<br>Jueves 11h30 – 13h30 |
| **Lugar** | Aula, laboratorio |
| **Tiempo planificado en el Sílabo** | 3 horas |

## 2. Objetivo(s) de la Práctica:

Diseñar e implementar un sistema interactivo de juego de luces con 6 LEDs de diferentes colores controlados por un pulsador. Aplicar conceptos de funciones, variables, bloques condicionales (if-else, switch-case) y bucles (for, while) para crear patrones de iluminación secuenciales que respondan a la entrada del usuario

## 3. Materiales y reactivos:

- 1 Protoboard
- 1 Arduino UNO
- 6 LEDs (Rojo
- Verde
- Azul
- Amarillo
- Blanco
- Naranja)
- 6 resistencias 220 ohmios

- 1 pulsador
- 1 Resistencia 10k ohmios
- Cables puente

## 4. Equipos y herramientas

- Computadora con Arduino IDE
- Componentes electrónicos
- Protoboard

## 5. Procedimiento / Metodología

# Algoritmo

```
ALGORITMO APE 02 - Juego de Luces Interactivo
INICIO
  LED_PINS <-- [2,3,4,5,6,7]
BOTON_PIN <-- 8   patronActual
<-- 0
  PARA cada pin EN LED_PINS: CONFIGURAR como SALIDA
  CONFIGURAR BOTON_PIN como ENTRADA
REPETIR     leerBoton()
    SEGUN patronActual HACER
      CASO 0: patronSecuencia()
      CASO 1: patronPersecucion()
      CASO 2: patronParpadeo()
      CASO 3: patronAleatorio()
      CASO 4: patronOnda()
    FIN SEGUN
  FIN REPETIR
FIN
 FUNCION leerBoton():
patronActual = aleatorio()
FIN FUNCION
```

## 6. Resultados esperados:

- Se obtuvo un circuito con 6 LEDs que logra ejecutar 5 patrones de iluminacion. Al presionar el switch-case se cambia secuencialmente entre: secuencia, carrusel, parpadeo, aleatorio y onda ascendente. El sistema implementa debounce correctamente y switch-case para selección
- Paso 1: Se monto un circuito con 6 LEDs en los pines D2-D7, cada uno con resistencia 220 ohmios a GND y conectar pulsador al pin D8 con resistencia pull-down 10k ohmios.

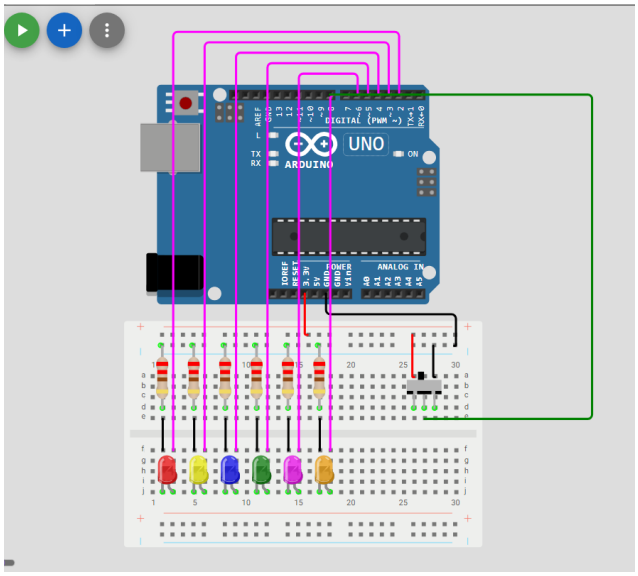


*Imagen 1: Circuito montado – Simulación*

- Paso 2: Se creo funciones para patrones: secuencia, persecución, parpadeo, aleatorio, Implementando una selección de patrones usando switch-case en la herramienta de simulación Wokwi (el código se encuentra en el GitHub)
- Paso 3: Usar bucles for y while para control de secuencias y temporización. El diagrama de conexión (físico/virtual)

```
const int LED_PINS[] = {2, 3, 4, 5, 6, 7};
const int NUM_LEDS = 6;
const int BOTON_PIN = 8;

int patronActual = 0;
const int NUM_PATRONES = 5;

int estadoBoton = LOW;
int estadoAnterior = LOW;
unsigned long tiempoUltimoDebounce = 0;
const unsigned long DELAY_DEBOUNCE = 50;


void setup() {
  for (int i = 0; i < NUM_LEDS; i++) {
    pinMode(LED_PINS[i], OUTPUT);
  }
  pinMode(BOTON_PIN, INPUT);
  randomSeed(analogRead(A0));
}

void loop() {
  leerBoton();

  switch (patronActual) {
    case 0: patronSecuencia();   break;
    case 1: patronPersecucion(); break;
    case 2: patronParpadeo();    break;
    case 3: patronAleatorio();   break;
    case 4: patronOnda();        break;
  }
}

```

```
void leerBoton() {
  int lectura = digitalRead(BOTON_PIN);

  if (lectura != estadoAnterior) {
    tiempoUltimoDebounce = millis();
  }

  if ((millis() - tiempoUltimoDebounce) > DELAY_DEBOUNCE) {
    if (lectura != estadoBoton) {
      estadoBoton = lectura;
      if (estadoBoton == HIGH) {
        patronActual = (patronActual + 1) % NUM_PATRONES;
        apagarTodos();
      }
    }
  }
  estadoAnterior = lectura;
}

void apagarTodos() {
  for (int i = 0; i < NUM_LEDS; i++) {
    digitalWrite(LED_PINS[i], LOW);
  }
}

void encenderTodos() {
  for (int i = 0; i < NUM_LEDS; i++) {
    digitalWrite(LED_PINS[i], HIGH);
  }
}

void patronSecuencia() {
  for (int i = 0; i < NUM_LEDS; i++) {
    leerBoton();
    digitalWrite(LED_PINS[i], HIGH);
    delay(150);
    digitalWrite(LED_PINS[i], LOW);
  }
}
```

```
void patronPersecucion() {

  for (int i = 0; i < NUM_LEDS; i++) {
    leerBoton();
    apagarTodos();
    digitalWrite(LED_PINS[i], HIGH);
    delay(120);
  }
  for (int i = NUM_LEDS - 2; i > 0; i--) {
    leerBoton();
    apagarTodos();
    digitalWrite(LED_PINS[i], HIGH);
    delay(120);
  }
}

void patronParpadeo() {
  encenderTodos();
  delay(300);
  apagarTodos();
  delay(300);
}

void patronAleatorio() {
  leerBoton();
  apagarTodos();
  int led = random(0, NUM_LEDS);
  digitalWrite(LED_PINS[led], HIGH);
  delay(200);
}

void patronOnda() {
  for (int i = 0; i < NUM_LEDS; i++) {
    leerBoton();
    apagarTodos();
    for (int j = 0; j <= i; j++) {
      digitalWrite(LED_PINS[j], HIGH);
    }
    delay(150);
  }
  for (int i = NUM_LEDS - 1; i >= 0; i--) {
    leerBoton();
    digitalWrite(LED_PINS[i], LOW);
    delay(150);
  }
}
```

- Paso 4: Generar el diagrama de conexión (físico/vitual)

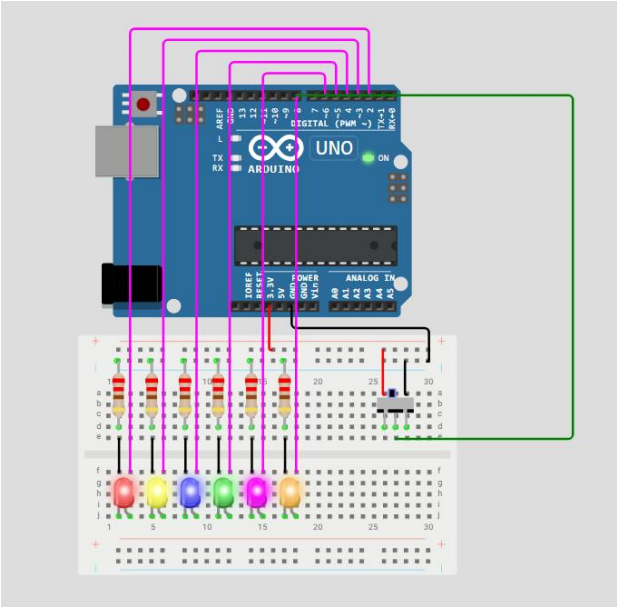


*Imagen 2: Diagrama virtual en wokwi – simulación*

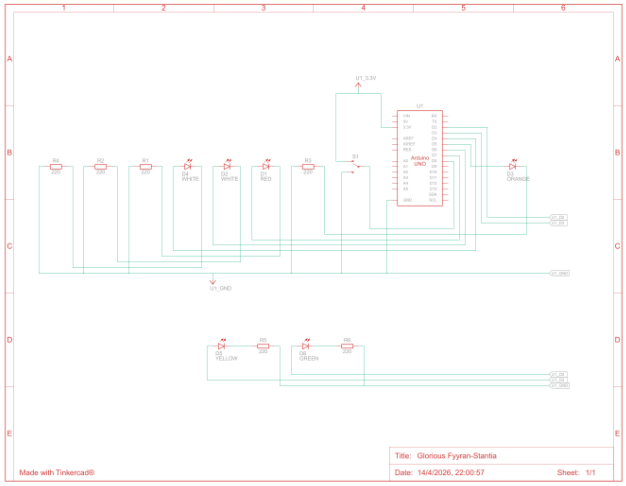


*Imagen 2: Diagrama Esquemático del circuito*

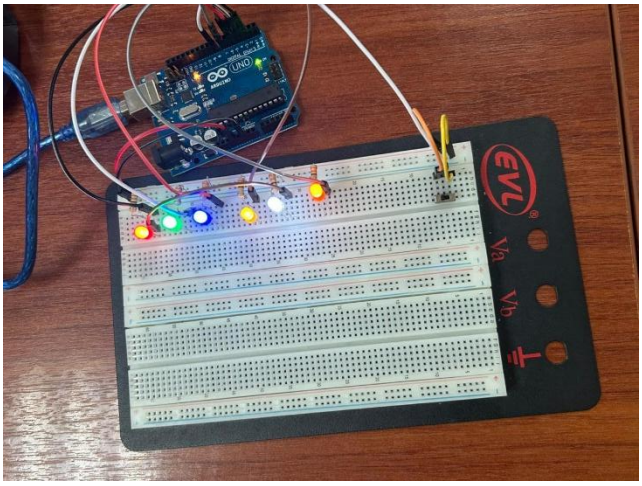

*Imagen 3: Simulación física del juego de leds*

- Proyecto funcional

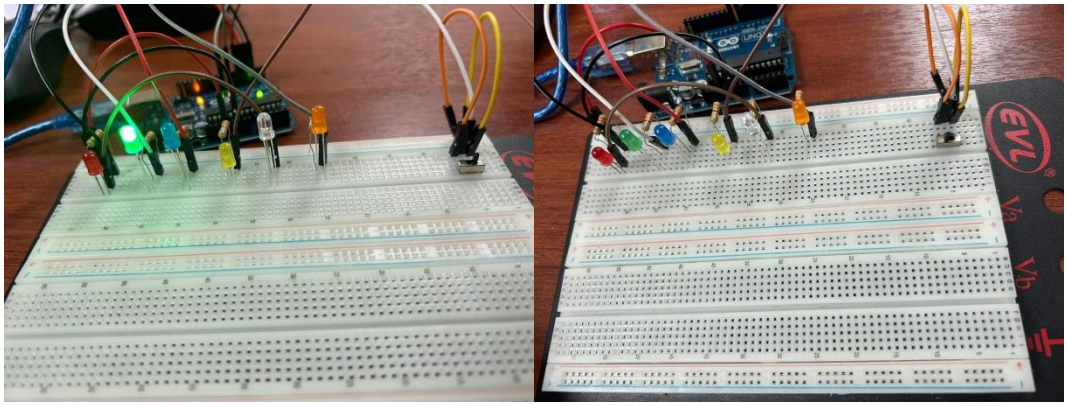

*Imagen 4 y 5: Simulación funcional*

Link de Wokwi: https://wokwi.com/projects/461298812991260673
Link de GitHub: https://github.com/vivinaCordova/Arquitectura_Microcontroladores.git

## 7. Preguntas de Control:

1. Diferencia entre bucles for y while y cuando usar cada uno.

   La diferencia entre los bucles for y while radica en su uso. El bucle for se utiliza cuando se conoce de antemano cuántas veces se va a repetir un bloque de código, ya que incluye en una sola estructura la inicialización, la condición y el incremento o decremento[1]. En cambio, el bucle while se utiliza cuando no se sabe exactamente cuántas veces se repetirá el ciclo, sino que depende de que una condición se cumpla.[1]

2. ¿Por qué se necesita debounce en el botón?

   El debounce es necesario porque los botones físicos no generan una señal limpia al ser presionados. En lugar de cambiar de estado una sola vez, producen múltiples cambios rápidos debido a rebotes eléctricos[1], [2]. Sin debounce, el sistema podría interpretar una sola pulsación como varias. El debounce permite filtrar estos rebotes y asegurar que se detecte una única pulsación real.[2]

3. Ventajas de switch-case sobre if-else anidados.

   El debounce es necesario porque los botones físicos no generan una señal limpia al ser presionados. En lugar de cambiar de estado una sola vez, producen múltiples cambios rápidos debido a rebotes eléctricos[1]. Sin debounce, el sistema podría interpretar una sola pulsación como varias. El debounce permite filtrar estos rebotes y asegurar que se detecte una única pulsación real.[2]

4. ¿Como modificaría para seleccionar patrones con dos botones?

   Para seleccionar patrones con dos botones, se puede asignar una función diferente a cada uno. Por ejemplo, un botón puede servir para avanzar al siguiente patrón y el otro para retroceder al anterior.[1] Es importante controlar los límites para evitar valores fuera de rango, haciendo que el contador vuelva al inicio o al final según corresponda[2].

5. ¿Qué es una máquina de estados finitos y como se aplicaria?

   Una máquina de estados finitos es un modelo de programación en el que un sistema puede encontrarse en un número limitado de estados y cambiar entre ellos en función de ciertos eventos o condiciones[2]. En la práctica, se utiliza para organizar el comportamiento de un programa, como en el caso de controlar diferentes modos de funcionamiento (por ejemplo, apagado, encendido y parpadeo), facilitando la estructura y comprensión del código[2].

## 8. Evaluación

no aplica

## 9. Declaración del uso de IA generativa

Se uso la IA generativa como nootebookIA, al cual se le cargo los papers consultados, los cuales sirvieron como una ayuda para responder dudas y las preguntas de control planteadas, lo que facilito una compresión mucho más clara de la actividad.

Por otra parte, se uso la IA de chatGPT y Claude IA, para ayudar a generar el código para la práctica, procurando tener una visión critica y responsable para la práctica. En esta actividad se hizo uso de la IA generativa, pero siempre procurando usarla de manera ética y responsable

## 10. Bibliografía

[1] Carlos. Valdivieso and R. M. . Solís, *Microprocesadores Fundamentos y Aplicaciones Carlos Valdivieso*. LATIn Project, 2018.

[2] C. Jarne *et al.*, "FUNDAMENTOS DE LOS CIRCUITOS SECUENCIALES."

## 10. Elaboración y Aprobación

| | | |
|---|---|---|
| **Elaborado por** | José Guamán **Docente** | |
| **Aprobado por** | Edison L Coronel Romero **Director de Carrera** | |